Artifact Narrative (Overall): Grazioso Salvare Animal Shelter Dashboard

Brief artifact description

Artifact name: Grazioso Salvare Animal Shelter Dashboard
Original course/context: CS-340 Advanced Programming Concepts
Original creation date: August 2024 (Module 4–6 final submission)

The original artifact was a Jupyter Notebook-based dashboard built with Dash, connecting directly to MongoDB (Austin Animal Center data). It supported filtering for rescue categories, displaying tabular results, and showing visualizations (map + breed analytics).

Why this artifact is included in my ePortfolio

I selected this artifact because it is a realistic, end-to-end application that benefits directly from professional enhancements. It is a strong vehicle for demonstrating growth across the three CS 499 categories:

- **Software design and engineering:** evolving a monolithic notebook into a structured, production-style architecture
- **Algorithms and data structures:** improving search and performance using workload-driven algorithm choices
- **Databases:** applying indexing, aggregation, and auditability to support scalability and decision making

The artifact also matches my specialization in **technical leadership** because it requires consistent trade-off decisions, stakeholder-focused communication, and evidence-based improvements.

What components showcase my skills (original vs enhanced)

Original strengths

- Functional UI and visualizations built quickly in Dash
- Clear problem focus: finding animals suitable for rescue training
- Direct use of a real dataset and operational queries

Enhanced components that showcase skills

- **Architecture:** three-tier structure (React frontend + FastAPI backend + MongoDB/Redis)
- **API design:** REST endpoints, standardized responses, auth boundaries
- **Security and compliance posture:** JWT + RBAC, rate limiting, security headers, audit logging
- **Performance:** caching to reduce repeated reads, search features tailored to user behavior

- **Database maturity:** compound/text indexes and aggregation pipelines for analytics
- **Testing:** unit/integration tests that support safe iteration

How the enhancements improved the artifact

The enhancements improved the artifact from a prototype suitable for coursework into a solution that better reflects industry expectations:

- **Maintainability:** modular services and cleaner separation of concerns
- **Scalability:** server-side pagination/filtering and caching
- **Usability:** autocomplete and fuzzy search to match real user input behavior
- **Auditability:** logging of operations for traceability and incident response
- **Professional communication:** narratives and portfolio organization that explain decisions and trade-offs clearly

Reflection: learning, challenges, feedback, and outcomes

What I learned

- Professional engineering is as much about *structure, evidence, and communication* as it is about implementation.
- Architectural decisions should be driven by maintainability and the ability to change safely.
- Performance improvements require clear assumptions about workload patterns and trade-offs.
- Auditability and validation are not "extras"—they are core properties of trustworthy systems.

Challenges I faced

- Translating notebook-style logic into a maintainable service architecture
- Managing the trade-offs introduced by caching (invalidation, TTL, fallbacks)
- Designing database indexes and aggregations aligned to real query patterns
- Ensuring the system remains testable and understandable after enhancements

How feedback was incorporated

Across milestones, I used feedback and iteration to shift emphasis toward professional expectations:

- Stronger modularity and clearer responsibilities between layers
- More testing and documentation to reduce ambiguity and improve reviewability
- More explicit trade-off discussion and evidence mapping to outcomes

Course outcomes met (and what remains)

- **Outcome 1 (collaboration):** addressed through a reviewable codebase structure and a written code review deliverable; further improvements could include PR templates and multi-contributor workflow evidence.
- **Outcome 2 (communication):** met through narratives, portfolio organization, and stakeholder-oriented explanation.
- **Outcome 3 (algorithmic solutions):** met through trie/autocomplete, fuzzy matching, caching strategy, and explicit trade-offs.
- **Outcome 4 (tools/techniques):** met through FastAPI/React, testing practices, Docker-oriented deployment approach, and production-style services.
- **Outcome 5 (security mindset):** met through authentication/authorization, audit logging, validation, rate limiting, and defensive defaults.

Overall, this single artifact provides a cohesive demonstration of growth across the program because it ties together engineering discipline, algorithmic decision making, database maturity, and security awareness within one realistic system.